

SEAL

WALLET SECURITY

v0.1

Your friendly wallet
security companion

feat. **THE RED GUILD**

Security Alliance

WALLET SECURITY

v0.1

Your friendly wallet
security companion

Security Alliance

Table of Contents

About this book	9
Help us improve	10
Human safety comes first	11
Introduction	12
Disclaimer	14
How this book is organized	15
Who holds the keys	17
Custodial	17
Non-custodial	18
The trade	19
What this book assumes	19
Split the pile	21
Hot	21
Cold	22
Small balances	23

When the number grows	24
Hardware	25
Buy it like it matters	26
Set it up once, slowly	27
What to look for	27
A second device	28
The seed	30
Offline or it is not a backup	31
What never happens	31
Splits are examples	32
Orgs and succession	33
Never sign blind	34
Ground truth	34
Simulate, then doubt it	35
Four checks on an EOA	36
Practice when it is free	36

Approvals	38
Exact amounts	38
Permit is not a login	39
The usual traps	39
Trades	40
Before you click	41
More than one key	42
M of N	43
Independence	44
How you operate	44
New surfaces	47
Account abstraction	47
EIP-7702	48
TEEs and encumbered keys	49
If it breaks	51
Seed or key exposed	51

Signer seat burned	52
Device looks wrong	53
This is the wrong book	53
After	54
If it is happening now	56
Before you sign.....	56
Seed may be exposed	57
Approval or permit looks wrong....	58
Hardware arrived wrong	59
Conclusion	60

ABOUT – SECURITY ALLIANCE

Security Alliance (SEAL) is a nonprofit organization dedicated to securing the future of cryptocurrency. Under the leadership of whitehat hacker **samczsun**, the organization designs security projects and coordinates across the crypto industry so that useful practices actually get used.

Learn more at securityalliance.org

ABOUT – THE RED GUILD

A non-profit organization, think-tank-alike, devoted to do security as a public good within the Ethereum ecosystem. Its work spans security research, responsible disclosure, education, open-source tooling, community awareness and more. It is best known for developing gamified learning experiences like **Damn Vulnerable DeFi**, and their recent platform the **Phishing Dojo**.

Learn more at theredguild.org

blog.theredguild.org · phishingdojo.com

About this book

This mini-book is a print adaptation of the Wallet Security framework from the **Security Frameworks**, an initiative by the **Security Alliance**. The web pages were written for a site. This edition is rewritten for a pocket book you can hand to someone who is about to hold keys.

The source this edition follows is the reviewed personal custody core: who holds keys, hot versus cold, hardware setup, seed handling, signing verification, and approvals. Software-wallet and hardware-wallet stub pages, the signing-schemes stub, and the tools

catalog stay on the website. Empty chapters make a worse companion than a short one.

Safe and Squads operator walk-throughs live in the Protocol Multisig companion. This book will not pretend to be that one.

This print cut changes order, cuts website navigation, and turns the same controls into reading you can do on a train. It does not invent new controls to fill gaps.

Help us improve

The live framework moves. If a control here is wrong, incomplete, or now

dangerous in your jurisdiction, fix it upstream so the site and the next printing match.

`github.com/
security-alliance/frameworks`

If you value this kind of resource, you can support the work at our website.

`securityalliance.org/donate`

Human safety comes first

No asset, key, policy, or evidence goal is worth more harm. If someone is being coerced, put the wrench down and open the Physical Security companion. This book is for the key. That book is for the person.

Introduction

Wallet security is control of keys and verification of every signature. Hardware and multisig fail if you sign attacker-controlled calldata because a website told you to.

Cryptography protects keys from math. It does not protect you from a fake dApp, a poisoned address, a photo of a seed, or a hardware wallet that arrived already set up.

This book is for people who will self-custody: beginners with a hot wallet, operators moving savings onto hardware, and anyone about to be the

one person who can spend. It is not a protocol treasury runbook.

Read it as a loop, not as a policy you recite once:

1. Know who can spend, including the custodian you thought you did not have.
2. Split spend from savings. Match value to exposure.
3. Put the pile on hardware you bought and verified.
4. Keep the seed offline. Treat digital exposure as full compromise.
5. Never sign blindly. The device screen is the transaction.

6. Cap approvals. Ignore mystery tokens.
7. Add independent people before one key is life-changing.

Disclaimer

This is not legal advice, and it is not a recovery service. It will not match every chain, family, or threat model. Some of the dollar figures, seed splits, and slippage ranges in later chapters are examples from the source, not defaults. Copying them blindly is how you get a setup an attacker can predict.

A 3-piece seed split, a 25th word, or a custom encryption scheme can lock

you out harder than it locks an attacker out. Design that kind of control with people who do this work. Do not invent it at 2am.

The contents are free on the Security Frameworks site. Suggest changes there.

How this book is organized

1. **Who holds the keys.** Custody is who can spend.
2. **Split the pile.** Hot for spend, cold for savings.
3. **Hardware.** Buy direct, verify, sign on the device.

4. **The seed.** Offline. Any leak is compromise.
5. **Never sign blind.** Origin, address, function, amount.
6. **Approvals.** Exact amounts, revoke, permits are not logins.
7. **More than one key.** Independent signers, or theater.
8. **New surfaces.** AA, 7702, TEEs.
Power with extra failure modes.
9. **If it breaks.** Move. Do not reuse.

The last chapter is four short cards for the moment you do not want to reread a section.

Who holds the keys

Custody is who holds the private keys. That person, or that company, can spend. Everything else is branding.

Custodial

A custodial wallet is an account. An exchange or a wallet company holds the keys. You log in. They can reset the password. They can also freeze you, get hacked, go insolvent, or hand the keys to someone with a warrant.

Use this when you are trading on that venue, or when you accept counter-

party risk for the recovery convenience. Do not keep long-term savings there because the app is pretty.

Non-custodial

A non-custodial wallet is a key. You hold it. Nobody can reset it. Nobody can help you if you lose it. dApps talk to it directly.

This book is for that case. If someone else holds the keys, their security program is the book, not this one.

The trade

- Custodial: recovery exists. So does a third party who can lose or seize the funds.
- Non-custodial: no counterparty. You are the backup.

Account abstraction and social recovery try to sit between those poles. They do not delete the question. Someone still has to be able to spend, and you still have to know who.

What this book assumes

You will self-custody something.
Maybe a small hot wallet. Maybe the

savings. Maybe a signer seat. The habit that scales is the same: unique secrets, careful signing, offline backups. Start there even if the balance is not “serious” yet. Serious arrives faster than the backup does.

Split the pile

Hot wallets maximize convenience and online exposure. Cold wallets reduce remote attack surface and add friction. Match the fund to the exposure. Do not keep savings on the same hot path as everyday minting.

Hot

A hot wallet lives on a phone, a browser extension, or a desktop that is on the network. Fine for spend. Fine for learning. Wrong for the pile.

Download only from the official site or

store listing you verified twice. Open source helps. Independent reviews help more if you are not going to read the code. Wallet Scrutiny and similar scorecards are starting points, not blessings.

Cold

A cold wallet keeps the private key off the internet while it signs. Hardware wallets are the usual form. Paper wallets and air-gapped software exist. Brain wallets exist as a way to lose funds to a guessable sentence.

Most hardware still talks to a computer, over USB, Bluetooth, QR, or an SD card. That does not make it hot.

The test is whether the key stays isolated from the infected laptop during signing.

Multisig and account abstraction are authorization models, not types of cold wallet. Either can be hot, cold, or mixed.

Small balances

If loss would sting and not wreck you, a hot wallet is the honest setup. Unique seed. Offline backup the same day you create it. Careful signing from day one.

A small balance is practice for a larger one. Do not learn seed handling after

the number gets interesting.

When the number grows

Move the bulk onto hardware. Keep a hot wallet for spend. The savings wallet does not need to mint, vote, or approve random contracts.

If one person losing a laptop would change a life, you are past “one hot wallet” even if the number still feels medium.

Hardware

A hardware wallet is a small computer that holds keys and shows you what you are about to sign. Used properly, it is the last line when the laptop is dirty. Used badly, it is an expensive way to rubber-stamp a phishing site.

The standalone “Hardware Wallets” page on the site is still a stub. The working guidance lives in the intermediate-funds page. That is what this chapter follows.

Buy it like it matters

Buy direct from the manufacturer. Not a marketplace. Not a conference booth giveaway. Not a reseller with a deal.

Use a name that is not your public one if you can, and a delivery point that is not your front door.

It must arrive uninitialized. No PIN. No seed card already filled in. No “we set it up for you.” Reject that device. Do not debug it. Do not “just reset it.”

Then verify authenticity with the manufacturer’s own procedure before you generate anything. Ledger, Trezor, GridPlus, and the others each have one. Use theirs.

Set it up once, slowly

Update firmware before you create keys. Set a PIN that is not your phone PIN. Longer is better. Generate the seed on the device. Do not import a seed from a computer.

Do a dry-run recovery with the device's own feature before you send funds. If you use a 25th-word passphrase, check it in a temporary session and confirm the address matches.

What to look for

A screen large enough to show the full destination and the amount. Clear signing when the vendor has it. PIN

lockouts. A secure element with attestation if you are buying that class of device. A vendor who publishes what broke last time.

Clear signing is a help. It is not a substitute for reading the screen. If the laptop and the device disagree, the laptop is lying. Reject.

A second device

For anything you would hate to recover from a seed in a hurry, keep a second hardware wallet with the same seed, tested, stored separately. Confirm both still sign. Monthly is enough if you actually do it.

Teams should not all buy the same brand. One firmware bug should not take a quorum.

The seed

The seed phrase is the wallet. Every key the device derives comes from it. Anyone who sees it can empty every account later, quietly, from another country.

If you even suspect it was seen, photographed, typed into a connected device, or copied to a clipboard, treat it as gone. Make a new wallet. Move the funds. Do not “keep an eye on it.”

Offline or it is not a backup

Write it down when the device shows it. Paper is a start. Metal survives fire and water better. Store copies in places that do not burn, flood, or get burglarized together.

Wallet vendors cannot recover it for you. That is the point.

What never happens

No photo. No cloud. No chat. No email to yourself. No plaintext file. No password manager. No “I will memorize it.” Nobody legitimate will ask for it. Conference USB sticks are not storage.

Do not travel with the full seed. Do not store every piece in the same drawer.

Splits are examples

The source describes a 3-piece overlap split: words 1–16, 9–24, and 1–8 plus 17–24, so any two pieces rebuild a 24-word seed. Shamir, vendor multi-share, and a 25th word exist too.

Those are examples, not a policy. A split you cannot rehearse is how you lock yourself out. A split stored with people who do not know the code word is a story, not a recovery plan. If you encrypt the backup, the passphrase becomes another seed. Losing that is permanent.

Orgs and succession

If this seed is a signer in a quorum, losing you should not lose the quorum. Geographic copies. Theft of one location must not rebuild the wallet. Operator loss must not either.

Write down who can open which location, with a code word, and test it. Next of kin needs a path that is not a tweet. A lawyer's sealed instructions beat a puzzle only you understood.

Revisit locations twice a year. Know where every copy is. If the primary device died tonight, could you recover before the rent is due?

Never sign blind

Never sign blindly. Prefer the hardware wallet screen and independently checked calldata over any web UI. A compromised front end can look perfect and still feed you a drain.

The key is not stolen. You handed the spend over.

Ground truth

The laptop lies. The phone lies. The dApp lies. The simulation can lie. The trusted display on the hardware wallet

is what you are authorizing. If that screen and the website disagree, reject.

Demand clear signing. If the wallet wants a blind hash, you are accepting a lot of risk. Sometimes a contract forces it. Then the question is whether this interaction is worth that.

Simulate, then doubt it

Run a simulator on anything that is not a boring transfer you already understand. Read the predicted balance changes. Then remember a simulation can show one thing and execute another. It is a filter, not a blessing.

Four checks on an EOA

1. **Origin.** The URL. Clones are cheap.
2. **Contract.** The “interacting with” address, from official docs or a labeled explorer page, not from chat.
3. **Function and parameters.** The name, the to / recipient / onBehalfOf field, the decimals.
4. **Fee.** A simple transfer with a huge gas bill is a smell.

Practice when it is free

Wise Signer and similar drills exist so the first time you read calldata is not

with rent money on the screen. Use them.

The detailed Safe and Squads verification flows are in the Protocol Multisig companion and on the site. The rule does not change there: hash, simulate, decode, then sign. Last signer executes only after that.

Approvals

Most fund losses are interaction mistakes, not stolen seeds. You signed something that let a contract pull. The wallet was fine. The spend was not.

Exact amounts

`approve()` gives a spender permission to move a token. Most dApps ask for unlimited. Give the amount this transaction needs. Revoke the rest later with `Revoke.cash` or the explorer's approval checker. Do that on a schedule, not after a scare.

Permit is not a login

EIP-2612 permit is an off-chain approval. No on-chain tx appears until someone else submits it. A fake “sign in” prompt with spender, value, nonce, and deadline is a permit. You are not logging in. You are loading the gun.

The usual traps

Address poisoning. A tiny transfer from an address that looks like yours, or like a recipient you use. Scam tokens in the history. Copy from an address book you maintain, not from “recent.”

Clipboard malware. It swaps the address you copied. Read the destination on the hardware screen, character by character, before you confirm.

Fake airdrops. Unknown tokens in the wallet. Claiming or swapping them is how `setApprovalForAll` gets signed. Ignore them.

Ice phishing. You sign a real approve to a spender you were talked into. The chain did what you asked. The social layer was the exploit.

Trades

Slippage of 5–10% on a liquid pair is how you get sandwiched. Tiny slippage

is how you burn gas on a fail. The source's 0.5–1% band is an example for liquid pairs, not a law.

Private relay routes (Flashbots Protect, MEV Blocker) hide the tx from public searchers. Set a deadline so a stuck tx cannot execute later at a worse price. If an aggregator hops through a token you did not pick, stop.

Before you click

Verify the contract. Simulate. Check the approval amount. Read the typed data. If you do not understand the message, do not sign it.

More than one key

One key is one person, one device, one bad day. A multisig raises the bar only when the signers, the devices, the places, and the verification habit are actually independent. A poorly run multisig is theater.

This chapter is the personal and small-team version. Protocol treasuries, timelocks, pause guardians, and Safe operator steps live in the Protocol Multisig companion. Use that book when the wallet is an institution.

M of N

You need M signatures out of N keys. 2-of-3 and 3-of-5 are the usual shapes. Avoid N-of-N. Losing one key should not brick the funds.

The source's numbers (at least three signers, about half required, more signers above large balances) are examples from protocol practice. Copy the idea, not the spreadsheet.

Every signer uses hardware. Dedicated address per multisig, even if it is only another index on the same device. Same seed still means one seed incident burns every seat.

Independence

Two keys on one laptop is one key.
Two devices in one apartment is one burglary. Two signers who share a password manager is one phish. Different brands, different cities, different people.

Include at least one person who will actually read calldata. A quorum of executives who tap Approve because the group chat said so is a hot wallet with extra steps.

How you operate

Dedicated channel for signing. Backup way to reach people. Out-of-band

check before adding or replacing a signer: video plus a signed message, not a DM.

Never copy destinations from explorer history or chat. Poisoned lookalikes are cheap. First-time recipients get a tiny bidirectional test. Address changes get the full re-verify, not an edit in the address book.

Anyone can create a Safe, add your address, and park a lookalike in your dashboard. Check the full address. Bookmark the real one. Ignore mystery Safes.

If a key might be burned, tell the others immediately. The remaining quorum replaces it. That is the feature.

Do not wait until you “know for sure.”

New surfaces

New account types add recovery and policy. They also add code you can get wrong. None of this is the default for a first wallet.

Account abstraction

ERC-4337 makes the account a contract. Guardians, spending limits, batched calls, gas paid by someone else. The cost is that bugs in that contract, in the EntryPoint, in a paymaster, or in a guardian set can spend like a key.

Threat-model the guardians the way you threat-model signers. Same city, same employer, same phish, and you built a committee that fails together. Use the canonical audited EntryPoint. Do not collect random modules.

Key rotation without moving funds is the actual prize if the implementation is sound. Audits are not optional.

EIP-7702

7702 lets an EOA delegate to contract code. Batching and sponsored gas show up. So does a phishing line: “upgrade your wallet.” If you sign a SetCode to the wrong contract, they can drain you.

Major wallets hardcode the only delegation target they will offer. Approve that in the wallet app. A website, an email, or a DM asking you to enable smart-account mode is a phish.

A delegation with chain ID 0 can be replayed on other EVM chains. Revoke by delegating to the zero address. Delegation does not save you if the private key is already stolen.

TEEs and encumbered keys

Encumbered wallets put policy around a key, often inside a TEE. Useful if you need rules on a chain that cannot express them. Not automatic trustlessness. Hardware bugs, attestation lies,

and operator compromise still exist. Researchers keep proving TEEs fail open. Start at TEE.fail before you bet a treasury on an enclave.

If you need distributed control, independent human signers are still the boring answer.

If it breaks

Speed beats dignity. A seed you “might” have shown is a seed you showed.

Seed or key exposed

New wallet, on a device you trust.
New seed, offline backup, then move.
Sweep tokens, NFTs, and anything that needs a separate transfer. Revoke approvals on the old address after the value is gone, from the new setup, not by lingering on the burned one.

Do not reuse the old seed, the old

device, or the old browser profile.
Familiar is how you get hit twice.

Signer seat burned

Tell the other signers on a channel that is not the one you fear. Remaining quorum replaces the address. If you cannot prove the old key is yours because it is gone, they verify you some other way: video, known social, whatever you documented in advance. Inventing that process during the incident is how a lookalike joins.

Device looks wrong

Stop signing. Do not factory-reset away the evidence until you have a replacement that actually works. Treat secrets that were on an unlocked phone as gone.

Hardware that was out of your sight, mailed oddly, or offered as a gift is not your hardware.

This is the wrong book

If the problem is a person with a wrench, a kidnapping, or a family threat, close this and open Physical Security. Moving funds while someone is

still in the room can make the violence worse. Life first.

If the problem is travel, hotel Wi-Fi, or a border laptop, the travel OpSec companion is closer.

After

Look after the human who got hit before you demand a timeline. Then write down what was actually used: the site, the approval, the device, the copy of the seed. Close that hole. Rotate anything that sat next to it. Tell the people who hold the other copies.

You do not get to keep the old setup because rebuilding is annoying. The

attacker already has that setup.

If it is happening now

These are prompts, not a local emergency plan. Read them before you need them.

Before you sign

1. Official URL. Not a search result.
Not a chat link.
2. Contract address matches the docs.
3. Function, destination, and amount
match what you meant.

4. Hardware screen agrees. If it does not, reject.
5. Approval is the amount for this tx, not unlimited.

Seed may be exposed

1. Assume it is gone. Do not watch and wait.
2. New wallet on clean hardware. New seed, written offline.
3. Move funds. Then revoke what you can.
4. Never reuse the old seed, device, or browser.
5. Tell anyone who held a copy.

Approval or permit looks wrong

1. Reject. Close the tab.
2. spender / value / deadline is not a login.
3. Ignore mystery tokens. Do not claim them.
4. Check remaining approvals from a machine you trust.
5. If value already moved, this becomes incident response, not a retry.

Hardware arrived wrong

1. Pre-set PIN or pre-written seed: do not use it.
2. Conference gift, marketplace, or reseller: same.
3. Stop. Buy direct. Verify authenticity.
4. Generate on the new device. Dry-run recovery.
5. Move funds only after the replacement signs.

Conclusion

Wallet security is not a brand of device. It is knowing who can spend, keeping the seed off the network, and reading the screen every time.

The less any one key can move, the less a phish pays. The less a contract is approved to pull, the less a drainer finds. The more independent people you put on the savings, the less one laptop matters.

None of this is a checklist you tick in January. Balances grow. New dApps appear. Someone starts storing seeds in photos again. Re-read the threat

when the number changes, when you become a signer, when a new account type shows up in a wallet prompt.

If something already happened, do not reuse the compromised path because it is familiar. Move. Rebuild from material that was never on that screen.

You will not get this perfect. The job is to make a blind signature expensive, recovery possible, and panic unnecessary. Keep the spend small. Keep the seed offline. Sign only what you can explain.

ACKNOWLEDGMENTS

This pocket book follows the Wallet Security framework. Current steward and top contributor is Piña (pinalikefruit). Reviewers in this snapshot include Coinspect and engn33r.

Current stewards for other frameworks:

AI Security. curtis0x, masterfung, munamwasi

Community Management. NFTDreww

DNS & Domain Registration Security.

Raiders

DevSecOps. Elliot

ENS. Ghadi Mhawej

Incident Management. EK, Nick K

Monitoring. Josep Bove

Operational Security. Seth Hallem

Physical Security. AllyPry

Safe Harbor. Dickson Wu

SEAL Certs. Dickson Wu, Isaac Patka

Security Testing. Patrick Collins

SOURCE SNAPSHOT

This cut follows the reviewed personal-custody pages of the Security Alliance Wallet Security framework. Website navigation, in-progress stubs, tool catalogs, and Safe / Squads operator walkthroughs are not in the book.

Branch develop

Commit c551872e2652

Commit date 2026-09-08

Build date 2026-09-14

Online frameworks.securityalliance.org

Framework authors in this snapshot

pinalikefruit, Coinspect, engn33r, njelich,
quillaudits

Check the site before you treat this as
current.

@ 2026 Security Alliance / The Red Guild

Security Frameworks copyright 2023–2026 Security Alliance and contributors. This pocket edition is Creative Commons Attribution-ShareAlike 4.0 International.

creativecommons.org/licenses/by-sa/4.0/

Share and adapt, including commercially, if you give credit, say what changed, link the license, and keep adaptations under the same license.

Gentium Book is SIL OFL 1.1. Latin Modern is GUST Font License.

Pocket edition – 2026

Published by Security Alliance (SEAL) · The Red Guild